

Cron at the center of your pipelines

Cron is not a Strategy. It's a Gamble. 🎰 🎲

The context

We all started with Cron. But Cron has no memory. It doesn't know if the last run failed. It doesn't know if the system crashed mid-task. WPipe is the "Cron for the 21st Century."

Why engineers shift to wpipe

- State Awareness: WPipe knows what finished and what didn't.
- Auto-Resume: Using SQLite WAL mode, it resumes from the last successful checkpoint.
- Parallel Execution: Why run sequentially when you can bypass the GIL?

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=10)
def job(data):
    return {"done": run(data["command"])}

pipe = Pipeline(pipeline_name="cron_pro", tracking_db="cron.db")
pipe.set_steps([job])

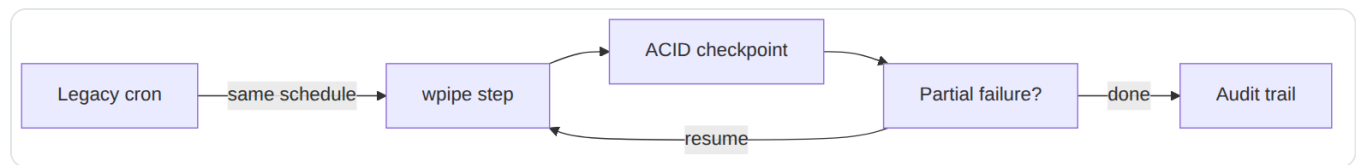
# Keep the schedule; gain checkpoints + audit.
```

At a glance

Aspect	Legacy / Cron	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Cron** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.